Step 3 — Add the Resource and the Prompt — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Two hints — work through them in order.

- Hint 1 — The pieces, and the decision that matters
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The pieces, and the decision that matters

The code is small. The point is choosing correctly which MCP building block each new thing is:

- `notes://all` is a **Resource**, not a third Tool. Ask one question: does it *change* anything, or cost real computation? No — it only hands back data to read. That's a Resource. A client that just wants to read your notes shouldn't go through a Tool call (which has its own approval model in most clients) to do it.
- `summarize_notes` is a **Prompt**, not a hardcoded string. It's a reusable, parameterized template. The server returns the *text of the prompt*, filled with the two dates — it never calls a model and never reads the database. Point it at the `notes://all` Resource for the actual notes.

The pieces:

- `src/db.py` → add `list_all_notes(path)` — a plain `SELECT id, text, created_at FROM notes ORDER BY id`.
- `server.py` → `@mcp.resource("notes://all")` on a no-argument function, and `@mcp.prompt()` on `summarize_notes(start_date, end_date)`. Give both a real docstring — a Resource's and a Prompt's descriptions matter to a client's UI the way a Tool's matters to a model.

`add_note` and `search_notes` do **not** change — they stay Tools, on purpose.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
# server.py (new pieces, below the two Tools)
from src.db import list_all_notes

@mcp.resource("notes://all")
def all_notes() -> str:
    """A read-only listing of every note currently stored, oldest first."""
    rows = list_all_notes(DB_PATH)
    if not rows:
        return "No notes yet."
    # your turn: build one line per row, e.g. "[1] Buy milk (timestamp)",
    # with a plain for-loop, and "\n".join(...) them

@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
    """A reusable template... returns the prompt text only."""
    # your turn: return a prompt string that mentions both dates and tells
    # the caller to use the notes://all Resource for the actual notes
    ...
```

Watch the empty-store case: the Resource should return `"No notes yet."`, not a silent empty string that looks like a bug. Test both with the Inspector (read `notes://all`, call `summarize_notes`), then add two integration cases: the resource lists a note, and the prompt text contains a date you passed in. Compare against the Solution.